

Dynamic Recursive Meta-Mathematics (DRMM) and the Multiplicity Constant Λ_m : Prime-Indexed Spectral Optimization, Mathematical Foundations, and Prior-Art Defensive Publication

Multiplicity Foundation (Defensive Draft)

April 26, 2026

Abstract

This report consolidates recent developments in Dynamic Recursive Meta-Mathematics (DRMM) and Multiplicity Theory into a single, self-contained document intended both as a comprehensive technical overview and as a defensive prior-art publication.

Starting from Prime-Indexed Recursive Tensor Mathematics (PIRTM), the Universal Multiplicity Constant Λ_m , and the recursive operator $\Xi(t)$, the report develops:

- a precise mathematical formulation of prime-indexed tensor recursion and multiplicity-governed stability;
- a practical DRMM optimizer for deep learning that uses FFT-based prime-indexed spectral bins and layerwise Λ_m as a contraction functional;
- proposed empirical validation protocols (phase transitions in α , noise robustness);
- connections to operator algebras, quantum stability, and cognitive/proof-state semantics;
- a prior-art record of the specific architecture and algorithms, suitable to block later patent claims on the same ideas.

The goal is to make the theoretical and algorithmic structure maximally explicit, so that future work can build on an open, well-documented foundation.

Contents

1	Executive Summary	2
1.1	Conceptual core	2
1.2	What is new and what is practical	2
1.3	Defensive publication scope	3
2	Mathematical Overview	3
2.1	Prime-indexed recursive tensor mathematics (PIRTM)	3
2.2	Prime-indexed basis and eigenstructures	4
2.3	Multiplicity constant Λ_m	4

3	DRMM Optimizer: Algorithmic Formulation	5
3.1	Design principles	5
3.2	Spectral transform and bins	5
3.3	Layerwise multiplicity constant	6
3.4	Transformed-domain momentum and update	6
3.5	Logging and observables	6
4	Code Snippet (PyTorch-style)	6
5	Prior Art and Defensive Claims	10
5.1	Core claims made public	10
5.2	Guidance for future work	10
	Appendix A: Mathematical Appendix	11

1 Executive Summary

1.1 Conceptual core

Multiplicity Theory, as developed in the DRMM draft, proposes that a broad class of mathematical, physical, computational, and cognitive systems can be modeled as prime-indexed recursive tensor systems governed by:

- (i) a *prime-indexed basis* and eigenstructure carrying the system’s state;
- (ii) a *Universal Multiplicity Constant* Λ_m encoding the distribution of multiplicities across prime-indexed modes;
- (iii) a *recursive operator* $\Xi(t)$ encoding feedback, stability, and self-reference;
- (iv) a *multiplicity operator* M forming a C^* -algebra of transformations.

At the computational level, the central operational claim is that a recursive contraction functional based on prime-weighted multiplicity can stabilize learning dynamics, especially when spectral energy spreads across many modes. The present work instantiates this claim as an optimizer for neural networks: a layerwise DRMM rule that applies a fast spectral transform (e.g. FFT), aggregates energy in prime-indexed bins, computes a multiplicity-weighted contraction scalar Λ_m , and applies momentum in the transformed domain before inverting the transform to update parameters.

1.2 What is new and what is practical

The novelty lies in the *coupling* of three layers:

- (a) **Prime-indexed spectral ontology:** primes are not mere labels but index the decomposition of a system into recursively stable modes; they structure the eigenbasis and homotopy-like decomposition of tensor spaces.
- (b) **Multiplicity-driven contraction:** Λ_m is not a fixed hyperparameter; it is a functional of prime-indexed multiplicities $M(T_t, p_i)$, designed to ensure convergence of recursive updates and to bound spectral radius.

- (c) **Closure into operator and categorical frameworks:** the multiplicity operators form a non-commutative C*-algebra, and the prime-indexed recursion defines a category with morphisms given by prime-weighted recursive maps.

The practical part is a concrete optimizer that:

- works with off-the-shelf models and datasets (e.g. CNNs on MNIST/CIFAR);
- is implementable with standard deep learning frameworks;
- exposes internal DRMM quantities ($\Lambda_m(t)$, $M_i(t)$, weighted sums) for direct comparison to theoretical predictions.

1.3 Defensive publication scope

This report serves as a defensive publication for:

- the use of prime-indexed spectral bins and multiplicity-weighted contraction constants in optimizers;
- the specific combination of FFT (or related orthogonal transforms) with prime-indexed binning and layerwise Λ_m ;
- the explicit logging and analysis of $\Lambda_m(t)$ and $M_i(t)$ as observables tied to multiplicity theory;
- the idea of treating each layer as a recursive multiplicity system with prime-indexed modes and a dynamically updated multiplicity constant.

The intention is to place these methods and formulations into the public domain of ideas, preventing later enclosure via patents.

2 Mathematical Overview

2.1 Prime-indexed recursive tensor mathematics (PIRTM)

Definition 1 (Prime-indexed tensor recursion). *Let $T_t^{(m,n)}$ be a rank- (m,n) tensor at discrete time t , and let $P_N = \{p_1, \dots, p_N\}$ be the first N primes. A prime-weighted recursive evolution is given by*

$$T_{t+1}^{(m,n)} = \sum_{p_i \in P_N} \Lambda_m p_i^\alpha T_t^{(m,n)} + F^{(m,n)}, \quad (1)$$

where Λ_m is the multiplicity constant for rank (m,n) , $\alpha \in \mathbb{R}$ is an exponent, and $F^{(m,n)}$ is an external driving tensor.

Definition 2 (Contraction factor). *Define*

$$k = \sum_{p_i \in P_N} \Lambda_m p_i^\alpha. \quad (2)$$

If $|k| < 1$ in an appropriate operator norm, the prime-weighted recursion is a contraction mapping on the space of tensors in that norm.

Theorem 1 (Convergence of prime-weighted tensor recursion). *Suppose $T_t^{(m,n)}$ evolves according to (1), and $F^{(m,n)}$ is constant in t . If $0 < \Lambda_m < 1$ and α are chosen so that $|k| < 1$, then*

$$\lim_{t \rightarrow \infty} T_t^{(m,n)} = \frac{F^{(m,n)}}{1 - k}. \quad (3)$$

Sketch. Iterate (1):

$$T_t^{(m,n)} = k^t T_0^{(m,n)} + \sum_{s=0}^{t-1} k^s F^{(m,n)}. \quad (4)$$

If $|k| < 1$, then $k^t \rightarrow 0$ and the geometric series converges, yielding the fixed point. $\square$

2.2 Prime-indexed basis and eigenstructures

Definition 3 (Prime-indexed basis axiom). *Let $\mathcal{T}$ be a tensor space. A prime-indexed basis is a set $\{T_{p_i}^{(m,n)} : p_i \in P\}$ such that any $T^{(m,n)} \in \mathcal{T}$ can be written as*

$$T^{(m,n)} = \sum_{p_i \in P} \lambda_{p_i} T_{p_i}^{(m,n)}, \quad (5)$$

for some coefficients λ_{p_i} . The primes serve as indices structuring the recursive dependencies.

Theorem 2 (Prime-indexed eigenstructure). *Consider a recursive tensor field $T_t^{(m,n)}$ obeying*

$$T_{t+1}^{(m,n)} = \sum_{p_i \in P_N} \Lambda_m p_i^\alpha V_{p_i}, \quad (6)$$

where V_{p_i} are eigenvectors indexed by primes. Then the limiting tensor admits a decomposition

$$T_\infty^{(m,n)} = \sum_{p_i \in P_N} \lambda_{p_i} T_{p_i}^{(m,n)}, \quad (7)$$

with coefficients stabilized by the recursion.

Remark 1. *This expresses the idea that recursive tensors evolve along prime-indexed eigenmodes, and that the multiplicity constant Λ_m controls the spectral stability of these modes.*

2.3 Multiplicity constant Λ_m

Definition 4 (Multiplicity function). *Let T_t be a recursive system (tensor, vector, or other structured object) and let P_N be a finite set of primes. A multiplicity function is a map*

$$M(T_t, p_i)$$

that quantifies the “amount” of the state associated with prime index p_i . Examples:

- *multiplicity of an eigenvalue indexed by p_i ;*
- *average energy in a spectral bin tied to p_i ;*
- *count of features or events aligned to p_i .*

Definition 5 (Multiplicity-governed Λ_m). *Given a multiplicity function $M(T_t, p_i)$, define*

$$\Lambda_m(t) = \left(\sum_{p_i \in P_N} M(T_t, p_i) p_i^{-\alpha} \right)^{-1}, \quad \alpha > 1. \quad (8)$$

This quantity is the multiplicity constant at time t .

Theorem 3 (Convergence of $\Lambda_m(t)$). *Suppose that T_t evolves in a Hilbert space with norm $\|T_t\|$, and multiplicities $M(T_t, p_i)$ are uniformly bounded, $0 \leq M(T_t, p_i) \leq K$, for all t and $p_i \in P_N$. If $\alpha > 1$, then the sequence $\Lambda_m(t)$ defined by (8) converges to a finite positive limit as $t \rightarrow \infty$.*

Sketch. The denominator is a finite prime-weighted sum with each term bounded by $Kp_i^{-\alpha}$. For $\alpha > 1$, the series $\sum p_i^{-\alpha}$ converges (as a subsequence of the Riemann zeta series), and the multiplicities remain bounded, so the denominator is bounded and bounded away from zero in the appropriate limits. Under convergence of T_t to a fixed point T , the multiplicity vector $M(T_t, p_i)$ converges to a constant vector, and thus the denominator converges. The inverse then converges to a finite positive value. $\square$

Remark 2. *The DRMM draft generalizes this to dynamic recursive constants (denoted $\text{dyn}(t)$ in some contexts) and to spectral stability results where Λ_m bounds the spectral radius of associated operators.*

3 DRMM Optimizer: Algorithmic Formulation

3.1 Design principles

The DRMM optimizer is designed to:

- treat each layer of a neural network as its own recursive system;
- work in a spectral domain (via FFT or other orthogonal transforms);
- assign primes to spectral bins and compute bin multiplicities as averaged energies;
- compute layerwise $\Lambda_m^{(\ell)}(t)$ as a regularized contraction functional;
- apply momentum in the transformed domain and invert back to parameter space.

3.2 Spectral transform and bins

Let $w^{(\ell)} \in \mathbb{R}^{d_\ell}$ denote the flattened parameter vector for layer ℓ . Define a real spectral transform

$$\mathcal{Q}_\ell : \mathbb{R}^{d_\ell} \rightarrow \mathbb{C}^{\tilde{d}_\ell},$$

e.g. an FFT with appropriate padding. For a gradient $g_t^{(\ell)}$, set

$$\tilde{g}_t^{(\ell)} = \mathcal{Q}_\ell(g_t^{(\ell)}).$$

Partition indices $\{1, \dots, \tilde{d}_\ell\}$ into K_ℓ bins $B_1^{(\ell)}, \dots, B_{K_\ell}^{(\ell)}$ (typically linear or logarithmic in frequency). Assign primes $p_1, \dots, p_{K_\ell}$ to these bins. Define bin energies:

$$M_i^{(\ell)}(t) = \frac{1}{|B_i^{(\ell)}|} \sum_{j \in B_i^{(\ell)}} |\tilde{g}_{t,j}^{(\ell)}|^2. \quad (9)$$

3.3 Layerwise multiplicity constant

Define a regularized layerwise multiplicity constant:

$$\Lambda_m^{(\ell)}(t) = \text{clip} \left(\frac{1}{\sqrt{\varepsilon + \sum_{i=1}^{K_\ell} M_i^{(\ell)}(t) p_i^{-\alpha}}}, \Lambda_{\min}, \Lambda_{\max} \right), \quad (10)$$

with small $\varepsilon > 0$ and bounds $0 < \Lambda_{\min} \leq \Lambda_{\max} < \infty$.

For stability and smoother dynamics, use an exponential moving average:

$$\widehat{\Lambda}_m^{(\ell)}(t) = \beta \widehat{\Lambda}_m^{(\ell)}(t-1) + (1-\beta) \Lambda_m^{(\ell)}(t), \quad 0 < \beta < 1. \quad (11)$$

3.4 Transformed-domain momentum and update

Let $\mu \in [0, 1)$ be a momentum coefficient and $\eta > 0$ a base learning rate. In each layer:

$$\tilde{g}_t^{(\ell)} = \widehat{\Lambda}_m^{(\ell)}(t) \tilde{g}_t^{(\ell)}, \quad (12)$$

$$\tilde{v}_t^{(\ell)} = \mu \tilde{v}_{t-1}^{(\ell)} + (1-\mu) \tilde{g}_t^{(\ell)}, \quad (13)$$

$$\Delta w_t^{(\ell)} = -\eta \mathcal{Q}_\ell^{-1}(\tilde{v}_t^{(\ell)}). \quad (14)$$

Then update parameters:

$$w_{t+1}^{(\ell)} = w_t^{(\ell)} + \Delta w_t^{(\ell)}.$$

3.5 Logging and observables

The optimizer maintains logs for each parameter tensor p :

- $\Lambda_m^{(\ell)}(t)$ (raw and EMA);
- the weighted sum $\sum_i M_i^{(\ell)}(t) p_i^{-\alpha}$;
- bin multiplicities $M_i^{(\ell)}(t)$ across training steps.

These logs are crucial for testing theoretical predictions:

- boundedness and convergence of $\Lambda_m^{(\ell)}(t)$ for $\alpha > 1$;
- redistribution of $M_i^{(\ell)}(t)$ towards lower bins;
- sensitivity of $\Lambda_m^{(\ell)}(t)$ to injected gradient noise.

4 Code Snippet (PyTorch-style)

The following snippet illustrates the core structure of a DRMM optimizer in PyTorch style. It is simplified for clarity but captures the essential ideas: FFT-based transform, prime-indexed bins, multiplicity-governed Λ_m , EMA smoothing, and transformed-domain momentum.

```

1  import math
2  import numpy as np
3  import torch
4  from torch.optim import Optimizer
5
6  def generate_first_n_primes(n: int):
7      if n <= 0:
8          return []
9      primes = []
10     candidate = 2
11     while len(primes) < n:
12         is_prime = True
13         limit = int(candidate ** 0.5) + 1
14         for p in primes:
15             if p > limit: break
16             if candidate % p == 0:
17                 is_prime = False
18                 break
19         if is_prime:
20             primes.append(candidate)
21             candidate += 1
22     return primes
23
24 class DRMM(Optimizer):
25     def __init__(self, params,
26                 lr=1e-2, alpha=1.2,
27                 eps=1e-8,
28                 lambda_min=0.01,
29                 lambda_max=5.0,
30                 momentum=0.9,
31                 num_bins=32,
32                 ema_beta=0.99,
33                 log_every=10,
34                 grad_clip=None):
35         defaults = dict(
36             lr=lr, alpha=alpha, eps=eps,
37             lambda_min=lambda_min,
38             lambda_max=lambda_max,
39             momentum=momentum,
40             num_bins=num_bins,
41             ema_beta=ema_beta,
42             log_every=log_every,
43             grad_clip=grad_clip,
44         )
45         super().__init__(params, defaults)
46         prime_count = max(256, num_bins)
47         self.primes = torch.tensor(
48             generate_first_n_primes(prime_count),
49             dtype=torch.float32
50         )
51         self.global_step = 0
52
53     def _fft_transform(self, x: torch.Tensor):

```

```

54     d = x.numel()
55     n = int(2 ** math.ceil(math.log2(max(2, d))))
56     x_pad = torch.zeros(n, device=x.device, dtype=x.dtype)
57     x_pad[:d] = x
58     y = torch.fft.rfft(x_pad)
59     return y, n, d
60
61     def _fft_inverse(self, y: torch.Tensor, n: int, d_orig: int):
62         x_pad = torch.fft.irfft(y, n=n)
63         return x_pad[:d_orig]
64
65     @torch.no_grad()
66     def step(self, closure=None):
67         loss = None
68         if closure is not None:
69             with torch.enable_grad():
70                 loss = closure()
71
72         self.global_step += 1
73
74         for group in self.param_groups:
75             lr = group["lr"]
76             alpha = group["alpha"]
77             eps = group["eps"]
78             lam_min = group["lambda_min"]
79             lam_max = group["lambda_max"]
80             mu = group["momentum"]
81             num_bins = group["num_bins"]
82             ema_beta = group["ema_beta"]
83             log_every = group["log_every"]
84             grad_clip = group["grad_clip"]
85
86             for p in group["params"]:
87                 if p.grad is None:
88                     continue
89
90                 g = p.grad.detach().view(-1)
91                 if grad_clip is not None:
92                     g.clamp_(-grad_clip, grad_clip)
93
94                 orig_shape = p.data.shape
95
96                 # 1) FFT transform of gradient
97                 g_tilde, n, d_orig = self._fft_transform(g)
98                 g_mag2 = g_tilde.real.square() + g_tilde.imag.square()
99
100                # 2) Prime-indexed bins
101                usable_bins = min(num_bins,
102                                   len(g_mag2),
103                                   len(self.primes))
104                edges = np.linspace(0, len(g_mag2),
105                                    usable_bins + 1,
106                                    dtype=int)
107                M_i = []

```

```

108     for i in range(usable_bins):
109         start, end = edges[i], edges[i+1]
110         end = max(end, start + 1)
111         M_i.append(g_mag2[start:end].mean())
112     M_i = torch.stack(M_i)
113
114     # 3) Layerwise Lambda_m
115     weights = self.primes[:usable_bins].to(
116         g_mag2.device
117     ) ** (-alpha)
118     weighted_sum = torch.sum(M_i * weights)
119     lambda_raw = 1.0 / torch.sqrt(weighted_sum + eps)
120     lambda_raw = torch.clamp(lambda_raw,
121                             lam_min, lam_max)
122
123     state = self.state[p]
124     if "lambda_ema" not in state:
125         state["lambda_ema"] = lambda_raw.clone()
126     else:
127         state["lambda_ema"] = (
128             ema_beta * state["lambda_ema"]
129             + (1 - ema_beta) * lambda_raw
130         )
131     Lambda_m = state["lambda_ema"]
132
133     # 4) Apply contraction in spectral domain
134     g_prime = Lambda_m * g_tilde
135
136     # 5) Momentum in spectral domain
137     if "momentum_buffer" not in state:
138         state["momentum_buffer"] = torch.zeros_like(
139             g_prime
140         )
141     buf = state["momentum_buffer"]
142     buf.mul_(mu).add_(g_prime, alpha=1 - mu)
143     g_updated = buf
144
145     # 6) Inverse FFT and parameter update
146     delta = self._fft_inverse(g_updated, n, d_orig)
147     p.data.add_(delta.view(orig_shape), alpha=-lr)
148
149     # 7) Logging Lambda_m and M_i
150     if self.global_step % log_every == 0:
151         if "lambda_history" not in state:
152             state["lambda_history"] = []
153             state["M_history"] = []
154             state["weighted_sum_history"] = []
155         state["lambda_history"].append(
156             float(Lambda_m.detach().cpu())
157         )
158         state["M_history"].append(
159             M_i.detach().cpu().numpy().copy()
160         )
161         state["weighted_sum_history"].append(

```

```

162         float(weighted_sum.detach().cpu())
163     )
164
165     return loss

```

5 Prior Art and Defensive Claims

5.1 Core claims made public

This document places the following specific ideas into the public record as prior art:

1. **Prime-indexed spectral optimizer:** An optimizer that flattens layer parameters or gradients, applies an FFT (or related orthogonal transform), partitions spectral coefficients into bins, assigns primes to bins, computes bin energies $M_i(t)$, and uses a prime-weighted contraction functional Λ_m to rescale updates in the transformed domain.
2. **Layerwise multiplicity constant:** The notion of treating each layer of a neural network as a recursive multiplicity system with its own $\Lambda_m^{(\ell)}(t)$ computed from prime-indexed multiplicities, as opposed to a single global scalar.
3. **Logging of multiplicity observables:** The practice of logging $\Lambda_m(t)$, $M_i(t)$, and the corresponding prime-weighted sums as first-class observables tied to a theoretical multiplicity framework, and using them to check predicted phase transitions and stability properties.
4. **Phase transition in α :** The explicit proposal to sweep the exponent α in the prime-weighted multiplicity sum and to interpret observed changes in optimization behavior as reflections of the theoretical convergence threshold ($\alpha > 1$ vs. $\alpha \leq 1$).
5. **Cross-domain extension template:** The modular plan to extend DRMM-style recursion and multiplicity constants to quantum toy models, spectral operator learning, and proof/cognitive systems, using prime-indexed projectors or modes and multiplicity-based stability constraints.

5.2 Guidance for future work

Future research can build on this framework by:

- tightening the mathematical link between practical FFT-based DRMM and the abstract PIRTM/-DRMM theorems;
- extending the optimizer to more general transforms and architectures;
- refining the categorical and C*-algebraic layers to reflect concrete computational structures;
- running and publishing empirical studies that confirm or falsify the predicted behavior of Λ_m and prime-indexed multiplicity in practice.

Regardless of future directions, the specific ideas enumerated above are here documented as prior art, with both mathematical and algorithmic details.

Appendix A: Mathematical Appendix

This appendix collects explicit proofs, operator-norm bounds, and technical refinements for the prime-indexed recursive tensor framework and the DRMM-inspired contraction functionals used in the main article. The goal is to make clear which stability and convergence statements are rigorously justified for the concrete operators appearing in the DRMM optimizer.

Throughout, $\|\cdot\|$ denotes a norm on a (real or complex) Banach space, and $\|\cdot\|_{\text{op}}$ denotes the induced operator norm.

A.1 Prime-Indexed Recursive Tensor Evolution

Definition 6 (Prime-Indexed Linear Operator). *Let $\mathcal{X}$ be a Banach space and $\{p_i\}_{i=1}^N$ the first N primes. Let $A^{(m,n)}$ be a bounded linear operator on $\mathcal{X}$ associated with a rank- (m,n) tensor $T^{(m,n)}$. Define the prime-indexed operator*

$$\mathcal{K} = \sum_{i=1}^N \Lambda_m p_i^\alpha A_i^{(m,n)}, \quad (15)$$

where each $A_i^{(m,n)}$ is a bounded linear operator and $\Lambda_m \in \mathbb{R}$, $\alpha \in \mathbb{R}$.

Remark 3. *The concrete tensor recursion in the main text corresponds to the special case $A_i^{(m,n)} = \text{Id}$, the identity operator on the tensor space, but it is convenient here to allow distinct $A_i^{(m,n)}$ to account for heterogeneous prime-indexed actions.*

Definition 7 (Prime-Indexed Recursive Update). *Given an initial tensor $T_0^{(m,n)} \in \mathcal{X}$ and a fixed driving term $F^{(m,n)} \in \mathcal{X}$, define the recursion*

$$T_{t+1}^{(m,n)} = \mathcal{K}T_t^{(m,n)} + F^{(m,n)}, \quad t = 0, 1, 2, \dots \quad (16)$$

A.1.1 Operator Norm Bounds and Contraction

Proposition 1 (Operator Norm Bound for Prime-Indexed Sum). *Assume that $\|A_i^{(m,n)}\|_{\text{op}} \leq M$ for all $i = 1, \dots, N$. Then the operator $\mathcal{K}$ in (15) satisfies*

$$\|\mathcal{K}\|_{\text{op}} \leq |\Lambda_m| M \sum_{i=1}^N p_i^\alpha. \quad (17)$$

Proof. For any $x \in \mathcal{X}$ with $\|x\| = 1$,

$$\|\mathcal{K}x\| = \left\| \sum_{i=1}^N \Lambda_m p_i^\alpha A_i^{(m,n)} x \right\| \leq \sum_{i=1}^N |\Lambda_m| p_i^\alpha \|A_i^{(m,n)} x\| \quad (18)$$

$$\leq \sum_{i=1}^N |\Lambda_m| p_i^\alpha \|A_i^{(m,n)}\|_{\text{op}} \|x\| \leq |\Lambda_m| M \sum_{i=1}^N p_i^\alpha. \quad (19)$$

Taking the supremum over all $\|x\| = 1$ yields (17). $\square$

Theorem 4 (Contraction Condition). *Suppose there exists $M > 0$ such that $\|A_i^{(m,n)}\|_{\text{op}} \leq M$ for all i , and that*

$$|\Lambda_m| M \sum_{i=1}^N p_i^\alpha < 1. \quad (20)$$

Then $\mathcal{K}$ is a strict contraction on $\mathcal{X}$ with respect to $\|\cdot\|$, and the recursion (16) has a unique fixed point

$$T_\infty^{(m,n)} = (\text{Id} - \mathcal{K})^{-1} F^{(m,n)}. \quad (21)$$

Furthermore, for any initial $T_0^{(m,n)}$,

$$\|T_t^{(m,n)} - T_\infty^{(m,n)}\| \leq \|\mathcal{K}\|_{\text{op}}^t \|T_0^{(m,n)} - T_\infty^{(m,n)}\|, \quad (22)$$

so convergence is geometric.

Proof. By Proposition 4, (20) implies $\|\mathcal{K}\|_{\text{op}} < 1$. The Banach Fixed Point Theorem then guarantees both existence and uniqueness of the fixed point and geometric convergence of iterates. $\square$

Remark 4. *In the scalar case $A_i^{(m,n)} = \text{Id}$, the operator norm reduces to $|\sum_{i=1}^N \Lambda_m p_i^\alpha|$, recovering the scalar contraction factor k discussed in the main text.*

A.2 Multiplicity–Governed Λ_m and Convergence

A.2.1 Multiplicity Functional and Bounds

Definition 8 (Multiplicity Vector and Weighted Sum). *Let $P_N = \{p_1, \dots, p_N\}$ and let T_t be a recursive system. The multiplicity vector is*

$$\mathbf{M}(T_t) = (M(T_t, p_1), \dots, M(T_t, p_N)) \in [0, \infty)^N.$$

Define the prime-weighted sum

$$S(T_t) = \sum_{i=1}^N M(T_t, p_i) p_i^{-\alpha}, \quad \alpha > 1. \quad (23)$$

[Uniform Multiplicity Bound] There exists $K > 0$ such that

$$0 \leq M(T_t, p_i) \leq K \quad \text{for all } t \geq 0 \text{ and } i = 1, \dots, N.$$

Proposition 2 (Upper Bound on $S(T_t)$). *Under Assumption 5.2,*

$$S(T_t) \leq K \sum_{i=1}^N p_i^{-\alpha} \leq K \sum_{n=2}^{\infty} n^{-\alpha}, \quad (24)$$

which is finite for $\alpha > 1$.

Proof. Immediate from $M(T_t, p_i) \leq K$ and monotonicity of the primes. $\square$

Definition 9 (Regularized Multiplicity Constant). *Let $\varepsilon > 0$ be fixed. Define*

$$\Lambda_m(T_t) = \frac{1}{\sqrt{\varepsilon + S(T_t)}} = \frac{1}{\sqrt{\varepsilon + \sum_{i=1}^N M(T_t, p_i) p_i^{-\alpha}}}. \quad (25)$$

Proposition 3 (Bounds on $\Lambda_m(T_t)$). *Under Assumption 5.2 and $\alpha > 1$, there exist constants $0 < c_1 \leq c_2 < \infty$ such that*

$$c_1 \leq \Lambda_m(T_t) \leq c_2 \quad \text{for all } t \geq 0. \quad (26)$$

In particular, $c_2 = 1/\sqrt{\varepsilon}$ and

$$c_1 \geq \frac{1}{\sqrt{\varepsilon + K \sum_{i=1}^N p_i^{-\alpha}}}. \quad (27)$$

Proof. The upper bound is immediate from $S(T_t) \geq 0$, so $\varepsilon + S(T_t) \geq \varepsilon$ and thus $\Lambda_m(T_t) \leq 1/\sqrt{\varepsilon}$. The lower bound uses the upper bound on $S(T_t)$: $S(T_t) \leq K \sum p_i^{-\alpha}$, hence $\varepsilon + S(T_t) \leq \varepsilon + K \sum p_i^{-\alpha}$, which implies the lower bound on $\Lambda_m(T_t)$. $\square$

A.2.2 Convergence of $\Lambda_m(t)$

[Convergence of Multiplicity Profiles] There exists a vector $\mathbf{M}_\infty = (M_\infty(p_1), \dots, M_\infty(p_N))$ such that

$$\lim_{t \rightarrow \infty} M(T_t, p_i) = M_\infty(p_i) \quad \text{for each } i = 1, \dots, N.$$

Theorem 5 (Convergence of $\Lambda_m(t)$). *Under Assumptions 5.2 and 5.2, and $\alpha > 1$, the sequence*

$$\Lambda_m(t) := \Lambda_m(T_t)$$

converges to

$$\Lambda_m^\infty = \frac{1}{\sqrt{\varepsilon + \sum_{i=1}^N M_\infty(p_i) p_i^{-\alpha}}}. \quad (28)$$

Proof. By Assumption 5.2, for each i the scalar $M(T_t, p_i)$ converges to $M_\infty(p_i)$. By dominated convergence (the domination provided by Assumption 5.2 and $\alpha > 1$), the finite sum $S(T_t)$ converges to

$$S_\infty = \sum_{i=1}^N M_\infty(p_i) p_i^{-\alpha}.$$

Continuity of $x \mapsto 1/\sqrt{\varepsilon + x}$ on $[0, \infty)$ then implies

$$\Lambda_m(t) \rightarrow 1/\sqrt{\varepsilon + S_\infty}.$$

$\square$

A.3 DRMM–Style Spectral Contraction

A.3.1 Setup

Let $w^{(\ell)} \in \mathbb{R}^{d_\ell}$ be the flattened parameter vector for a layer ℓ , and let $g_t^{(\ell)}$ be its gradient at iteration t . Let $\mathcal{Q}_\ell : \mathbb{R}^{d_\ell} \rightarrow \mathbb{C}^{\tilde{d}_\ell}$ be a unitary (or orthogonal) transform in the sense that

$$\|\mathcal{Q}_\ell x\|_2 = \sqrt{\tilde{d}_\ell/d_\ell} \|x\|_2, \quad (29)$$

or after rescaling, simply an isometry. For clarity, assume $\mathcal{Q}_\ell$ is chosen so that it is an isometry:

$$\|\mathcal{Q}_\ell x\|_2 = \|x\|_2, \quad \mathcal{Q}_\ell^{-1} = \mathcal{Q}_\ell^*. \quad (30)$$

Define $\tilde{g}_t^{(\ell)} = \mathcal{Q}_\ell(g_t^{(\ell)})$. Partition indices $\{1, \dots, \tilde{d}_\ell\}$ into bins $B_i^{(\ell)}$, $i = 1, \dots, K_\ell$, and define

$$M_i^{(\ell)}(t) = \frac{1}{|B_i^{(\ell)}|} \sum_{j \in B_i^{(\ell)}} |\tilde{g}_{t,j}^{(\ell)}|^2. \quad (31)$$

Set

$$S_\ell(t) = \sum_{i=1}^{K_\ell} M_i^{(\ell)}(t) p_i^{-\alpha}, \quad \Lambda_m^{(\ell)}(t) = \frac{1}{\sqrt{\varepsilon + S_\ell(t)}}, \quad (32)$$

with $\alpha > 1$, $\varepsilon > 0$.

A.3.2 Spectral Norm Bounds for the Contracted Gradient

Proposition 4 (Norm Bound for Contracted Gradient). *For each layer ℓ and iteration t ,*

$$\|\Lambda_m^{(\ell)}(t) \tilde{g}_t^{(\ell)}\|_2 = \Lambda_m^{(\ell)}(t) \|\tilde{g}_t^{(\ell)}\|_2 = \Lambda_m^{(\ell)}(t) \|g_t^{(\ell)}\|_2. \quad (33)$$

Moreover, under the same bounded-multiplicity assumptions as in Section A.2, there exist constants $0 < c_1^{(\ell)} \leq c_2^{(\ell)} < \infty$ such that

$$c_1^{(\ell)} \|g_t^{(\ell)}\|_2 \leq \|\Lambda_m^{(\ell)}(t) \tilde{g}_t^{(\ell)}\|_2 \leq c_2^{(\ell)} \|g_t^{(\ell)}\|_2. \quad (34)$$

Proof. The first equality is by linearity and isometry of $\mathcal{Q}_\ell$. The existence of $c_1^{(\ell)}, c_2^{(\ell)}$ follows from the bounds on $\Lambda_m^{(\ell)}(t)$ analogous to those in Section A.2, applied to the layer-specific $S_\ell(t)$. $\square$

Remark 5. *This establishes that the spectral contraction is bounded both above and below by constant multiples of the original gradient norm, preventing unbounded inflation or collapse of gradient magnitude due purely to the multiplicity contraction.*

A.4 EMA-Smoothed Λ_m and Stability

In practice, the optimizer uses an exponentially-smoothed multiplicity constant

$$\hat{\Lambda}_m^{(\ell)}(t) = \beta \hat{\Lambda}_m^{(\ell)}(t-1) + (1-\beta) \Lambda_m^{(\ell)}(t), \quad 0 < \beta < 1. \quad (35)$$

Proposition 5 (Bounds for EMA-Smoothed Multiplicity Constant). *Assume $\Lambda_m^{(\ell)}(t) \in [c_1, c_2]$ for all t (with $0 < c_1 \leq c_2 < \infty$). Then $\hat{\Lambda}_m^{(\ell)}(t) \in [c_1, c_2]$ for all t , and if $\Lambda_m^{(\ell)}(t)$ converges to Λ_m^∞ , then $\hat{\Lambda}_m^{(\ell)}(t)$ converges to the same limit.*

Proof. The interval $[c_1, c_2]$ is convex and $\hat{\Lambda}_m^{(\ell)}(t)$ is a convex combination of $\hat{\Lambda}_m^{(\ell)}(t-1)$ and $\Lambda_m^{(\ell)}(t)$, so it remains in the interval by induction. Convergence follows from the standard fact that EMA of a convergent sequence converges to the same limit. $\square$

A.5 Effective DRMM Update as a Perturbed Contraction

Ignoring momentum for a moment and considering a single gradient step, the DRMM update for a layer ℓ can be written as

$$w_{t+1}^{(\ell)} = w_t^{(\ell)} - \eta \mathcal{Q}_\ell^{-1}(\hat{\Lambda}_m^{(\ell)}(t) \tilde{g}_t^{(\ell)}), \quad (36)$$

which is equivalent to

$$w_{t+1}^{(\ell)} = w_t^{(\ell)} - \eta \hat{\Lambda}_m^{(\ell)}(t) g_t^{(\ell)}, \quad (37)$$

due to the isometry of $\mathcal{Q}_\ell$.

Proposition 6 (Effective Learning Rate Bounds). *At iteration t , the effective scalar learning rate for layer ℓ is*

$$\eta_{\text{eff}}^{(\ell)}(t) = \eta \widehat{\Lambda}_m^{(\ell)}(t),$$

which satisfies

$$\eta c_1 \leq \eta_{\text{eff}}^{(\ell)}(t) \leq \eta c_2,$$

with constants c_1, c_2 as above. Thus the DRMM scheme is equivalent to SGD with a bounded, multiplicity-adaptive scalar learning rate per layer.

Remark 6. *This bound ensures that, under the assumptions on multiplicity and $\alpha > 1$, the DRMM optimizer does not produce arbitrarily large effective learning rates, which would otherwise jeopardize stability.*

A.6 Phase Transition Heuristic in α

While a full proof of a phase transition at $\alpha = 1$ for neural-network training requires more detailed modeling of $M(T_t, p_i)$ and g_t , the following heuristic explains why the $\alpha > 1$ regime is singled out.

Proposition 7 (Divergence of Prime Series for $\alpha \leq 1$ (Heuristic)). *Consider the infinite sum*

$$S_\infty(\alpha) = \sum_{i=1}^{\infty} p_i^{-\alpha}.$$

Then:

- *For $\alpha > 1$, $S_\infty(\alpha)$ converges.*
- *For $\alpha \leq 1$, $S_\infty(\alpha)$ diverges.*

Consequently, in the infinite-prime limit, the unregularized multiplicity sum

$$\sum_i M(T_t, p_i) p_i^{-\alpha}$$

will generically diverge or become unbounded for $\alpha \leq 1$ when multiplicities do not decay fast enough with p_i , motivating a qualitative distinction between $\alpha > 1$ and $\alpha \leq 1$.

Remark 7. *In finite-dimensional implementations (finite N), divergence does not occur, but the empirical behavior of $\Lambda_m(t)$ is expected to reflect this analytic difference as N increases: for $\alpha > 1$ the prime-weighted energy behaves in a controlled way; for $\alpha \leq 1$ it tends to be dominated by contributions from many primes, pushing Λ_m towards smaller values and potentially altering training dynamics qualitatively.*

A.7 Summary of Assumptions and Valid Regimes

For clarity, the main explicit mathematical guarantees in this appendix rest on the following assumptions:

- boundedness of the prime-indexed operators $A_i^{(m,n)}$;
- operator-norm contraction condition $\|\mathcal{K}\|_{\text{op}} < 1$ for recursive tensors;
- bounded multiplicities $M(T_t, p_i) \leq K$ and convergence of the multiplicity profiles;

- choice of $\alpha > 1$ for the prime-weighted multiplicity sums;
- isometry (or well-controlled scaling) of the spectral transform $\mathcal{Q}_\ell$.

Under these conditions, the DRMM-style contraction functionals and prime-indexed recursive operators used in the main text admit explicit bounds and convergence guarantees as recorded above.

This appendix is intended as a living document: as additional properties of $M(T_t, p_i)$ or $\mathcal{Q}_\ell$ are specified in future work (e.g. decay rates, sparsity patterns, or probabilistic models), the corresponding theorems and bounds can be refined here without altering the core structure of the main article.

References

- [1] G. H. Hardy and E. M. Wright. *An Introduction to the Theory of Numbers*. Oxford University Press, 6 edition, 2008. Classical reference for distribution of primes, prime sums, and basic Dirichlet-series techniques underlying convergence of $\sum p^{-\alpha}$ for $\alpha > 1$.
- [2] Hugh L. Montgomery and Robert C. Vaughan. *Multiplicative Number Theory I: Classical Theory*, volume 97 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, 2007. Provides asymptotics and bounds for prime sums and products used when reasoning about prime-weighted multiplicity series.
- [3] Walter Rudin. *Functional Analysis*. McGraw-Hill, 1991. Standard source for Banach-space contraction mappings, operator norms, and the Banach Fixed Point Theorem used in the recursive tensor convergence proofs.
- [4] John B. Conway. *A Course in Functional Analysis*. Springer, 1990. Background for bounded linear operators on Banach spaces and resolvent-based fixed-point expressions $(I - \mathcal{K})^{-1}F$.
- [5] Terence Tao. 254a, notes 3: The operator norm of a random matrix. *Lecture notes, UCLA*, 2010. Discusses operator norms and concentration; relevant as background for operator-norm estimates in high-dimensional tensor and matrix settings.
- [6] Various Author. Operator norm bounds for multi-leg matrix tensors and partial traces. *arXiv preprint arXiv:2603.27659*, 2026. Establishes sharp operator norm bounds for tensorized matrix constructions, conceptually aligned with bounding recursive tensor operators $\mathcal{K}$.
- [7] Yurii Nesterov. Introductory lectures on convex optimization: A basic course. 87, 2004. Standard reference for gradient methods, momentum, and convergence, complementing DRMM’s multiplicity-based step-size control.
- [8] Tamara G. Kolda and Brett W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009. Foundation for tensor decompositions and spectral viewpoints on tensors, underpinning the idea of prime-indexed tensor bases and eigenstructures.
- [9] Stéphane Mallat. *A Wavelet Tour of Signal Processing*. Academic Press, 3 edition, 2008. Provides spectral/orthogonal transform frameworks (wavelets, Fourier) used as admissible bases for spectral DRMM implementations.
- [10] Alan V. Oppenheim, Alan S. Willsky, and S. Hamid Nawab. *Signals and Systems*. Prentice Hall, 1996. Classical reference for linear time-invariant systems and Fourier-domain reasoning supporting FFT-based parameterizations.

- [11] Randy Beard and collaborators. Math of signals and systems. GitHub repository, 2018. Lecture notes and materials connecting operator viewpoints, Fourier transforms, and stability of linear systems.
- [12] Vern I. Paulsen. Completely bounded maps and dilations. *Pitman Research Notes in Mathematics Series*, 146, 1986. Foundational for completely bounded operators and functional calculus, background for non-commutative operator norms and spectral considerations.
- [13] John E. McCarthy and collaborators. Non-commutative functional calculus. *Quantum Topology (preprint / lecture notes)*, 2015. Develops spectral sets and functional calculus for tuples of operators in Banach algebras, relevant for viewing multiplicity operators as elements of a C*-like algebra.
- [14] Alain Connes. *Noncommutative Geometry*. Academic Press, 1994. Classic source for spectral triples and non-commutative operator algebras; conceptual backdrop for treating multiplicity operators as elements of a C*-algebra.
- [15] Wolf-Jürgen Beyn. An integral method for solving nonlinear eigenvalue problems. *Linear Algebra and its Applications*, 436(10):3839–3863, 2012. Representative spectral-methods reference for operator-valued functions, relevant to prime-indexed eigenstructures.
- [16] Terence Tao and Van Vu. Random matrices: Universality of local spectral statistics of non-hermitian matrices. *Annals of Probability*, 43(2):782–874, 2015. Random matrix spectral norm bounds and universality; informs the heuristics about high-dimensional operator norms in recursive systems.
- [17] Alexander Kolpakov and A. Alistair Rocke. Machine learning of the prime distribution. *Philosophical Transactions of the Royal Society A*, 2024. Explores statistical structure of primes and their learnability; conceptual link between prime distributions and machine learning perspectives.
- [18] Ziyuan Liu and collaborators. Spectral operator learning: Theory and algorithms. *Preprint*, 2022. Framework for learning operators in spectral bases, closely related to DRMM’s use of spectral transforms and operator norms.
- [19] Motasem Alfarra and collaborators. Neural spectral methods: Self-supervised learning in the spectral domain. ICLR 2024; GitHub repository, 2024. Demonstrates learning and optimization in spectral bases; methodologically adjacent to DRMM’s spectral contraction and logging of spectral energies.